

4_Memory

Moore's law

"... the observation that the number of transistors in a dense integrated circuit doubles approximately every two years."

Dennard scaling

"... as transistors get smaller their power density stays constant, so that the power use stays in proportion with area."

Multithreading

- Sometimes you can execute multiple operations at the same time
- How much the performance is improved is calculated using Amdahl's law

Pitfall: Amdahl's Law

Execution time after improvement =
$$\frac{\text{affected execution time}}{\text{amount of improvement}} + \text{execution time unaffected}$$

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}$$

- The performance will usually improve greatly adding threads until one point
- Tasks that can be divided cleanly and split off to threads of execution are called **embarrassingly parallel - this is the ideal scenario**
- In practice, not all tasks are fit for parallelization and introducing threads comes with its own overhead
- You as a programmer can set a lock, which claims exclusive ownership over some memory to some core

How does multicore work?

- When your computer starts you only have one core (which is called the "bootstrap core"), sends an inter-process interrupt to the other cores to "wake them up" and those cores receive the memory addresses where they need to start executing code

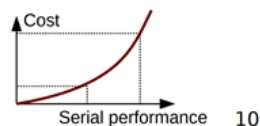
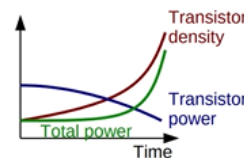
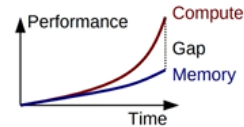
Explanation of the whole process:

```
computer boots: core 0 processor 0 - the bootstrap core - starts executing code at 0xffffffff0.
all other cores are "sleeping", in a Wait-for-SIPI state.
bootstrap core can send inter-processor interrupt (IPI) - a startup IPI (SIPI) -
over the advanced programmable interrupt controller (APIC).
the SIPI contains the address from which that core should start executing code.
this code can therefore be a kernel (instance specific to that core).
this core's kernel gets context-switched into at regular intervals (just like bootstrap-core).
this core's kernel can then schedule another thread to run.
since all cores' kernels share memory, they can coordinate (e.g. which thread runs on what kernel).
(they can announce to each other (by updating memory) that they are operational).
if a thread (on a core) wants to start a new thread, it makes a SYSTEM CALL;
the OS (on that core) takes care of the rest
(creating the thread in mem, which can then be picked up by any core's kernel for execution).
```

Limitations in scaling performance and memory

Technology evolution

- Memory wall
 - ♦ Memory speed does not increase as fast as computing speed
 - ♦ Harder to hide memory latency
- Power wall
 - ♦ Power consumption of transistors does not decrease as fast as density increases
 - ♦ Performance is now limited by power consumption
- ILP wall
 - ♦ Law of diminishing returns on Instruction-Level Parallelism
 - ♦ Pollack rule: $\text{cost} \approx \text{performance}^2$



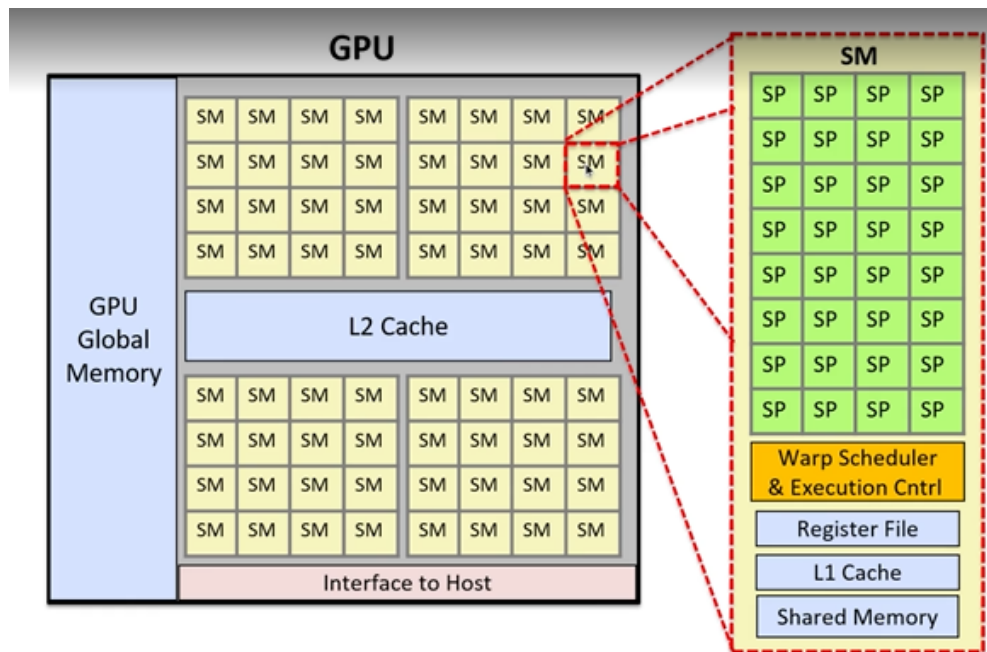
Heterogenous architectures

- Idea: combine architectures optimized for specific purposes and use the right one for the right job
- Example: CPU + GPU

- CPU is fast for sequential problems ("latency-optimized") but has low efficiency on parallel problems
- GPU is fast for parallel problems ("throughput-optimized") but slow for sequential problems

GPU Architecture

- Has a lot of cores
- The execution model is single instruction / multiple thread (SIMD)
- Has several streaming multiprocessors (SM) - each has an execution unit and memory and several scalar processors (SP)



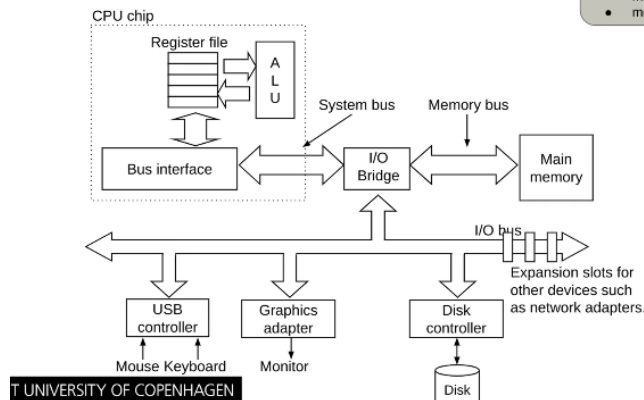
- Most popular GPU architecture is CUDA

lecture 4 actually starts here

- The CPU is connected to the memory by the I/O bridge

- I/O bus - collection of parallel wires that carry address, data and control signals
- Buses are usually shared by multiple devices

Computers are more like this



What happens when you read from a disk

- The instruction running the CPU will initiate the read by writing a command, logical block number and destination memory address to a port associated with disk controller
- Disk controller receives all of that information, reads the data and writes it directly to memory by performing a direct memory access (DMA) - the CPU is not involved in this
- When the transfer completes, the disk controller notifies the CPU with an interrupt
- So basically the memory has an API for reading and writing data

Locality

- The disk is very far away from the CPU (approx. 100k - 100mill clock cycles away)
- RAM is ~100 clock cycles away unless you have cache
- If you want CPI 1 (clock per instruction) you need to have the data in cache

- The principle of locality is: **use data and instructions with addresses equal or near those used recently**
- This is useful because reading from memory is a costly operation, but if we can predict memory and disk access then we can have data at hand before it is needed (ex prefetch)
- There are 2 types of locality

Locality types:

- **Temporal locality:**
recently referenced items are likely to be referenced again in the near future
- **Spatial locality:**
items with nearby addresses tend to be referenced close together in time

- Examples

```
sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];
return sum;
```

Good spatial locality

```
int sum_array_rows(int a[M][N])
{
    int i, j, sum = 0;

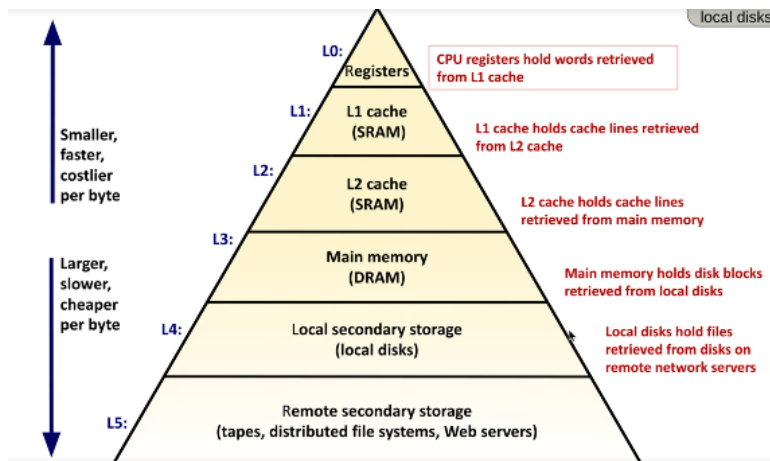
    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

```
int sum_array_cols(int a[M][N])
{
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

- `sum_array_rows` has better locality because the arrays will be one after each other
- "row major order" - the way a 2 dimensional array is stored in memory is that each array is stored one after the other, after the first array immediately comes the second array and so on

Memory hierarchy



Cache

- Small, fast storage layer that stores frequently accessed data to improve performance
- Allows the CPU to access frequently used items faster than retrieving them from main memory which is slow
- Accessing items from the cache is slightly slower than accessing registers (which is the fastest), but much faster than accessing main memory (RAM)
- **Levels of Cache**
 - **L1 Cache:** Closest to the CPU core; very small and extremely fast.
 - **L2 Cache:** Larger than L1 but slightly slower; shared by multiple cores in some processors.
 - **L3 Cache:** Even larger, shared across all cores, but slower than L1 and L2.
- Programs try to access memory as high as possible in the hierarchy (as fast as possible) (this is called "locality")

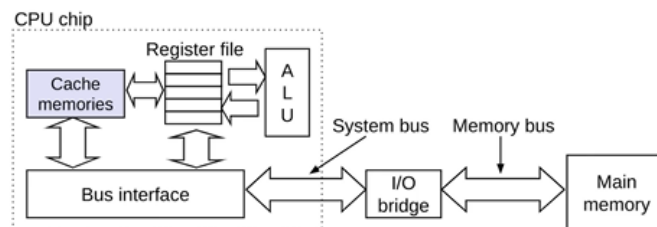
Cache misses

- **Cache miss** - if the CPU needs data from a line that currently isn't in the cache the CPU takes that line from main memory and puts it into the cache, evicting a previous line from the cache
 - Placement policy - determines where the new cache line goes

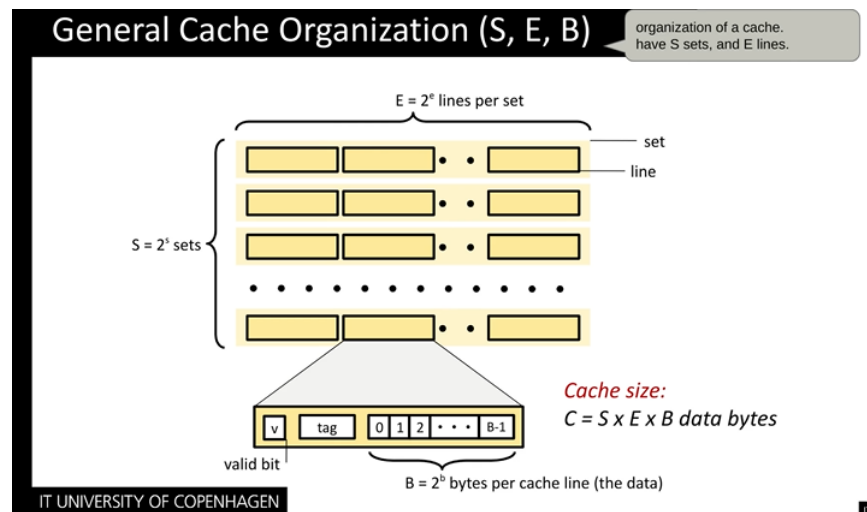
- Replacement policy - determines which line gets evicted
- **Types of cache misses**
 - **Cold (compulsory) miss** - the miss occurs because the cache is empty
 - **Conflict miss** - multiple elements map to the same line in the cache
 - Memory addresses are mapped to cache blocks based on the formula:

$$\text{Cache Block Index} = (\text{Memory Address}) \bmod (\text{Number of Cache Blocks})$$
 - **Capacity miss** - happens when there are more active cache lines than the cache capacity

Typical system structure:



Cache organisation

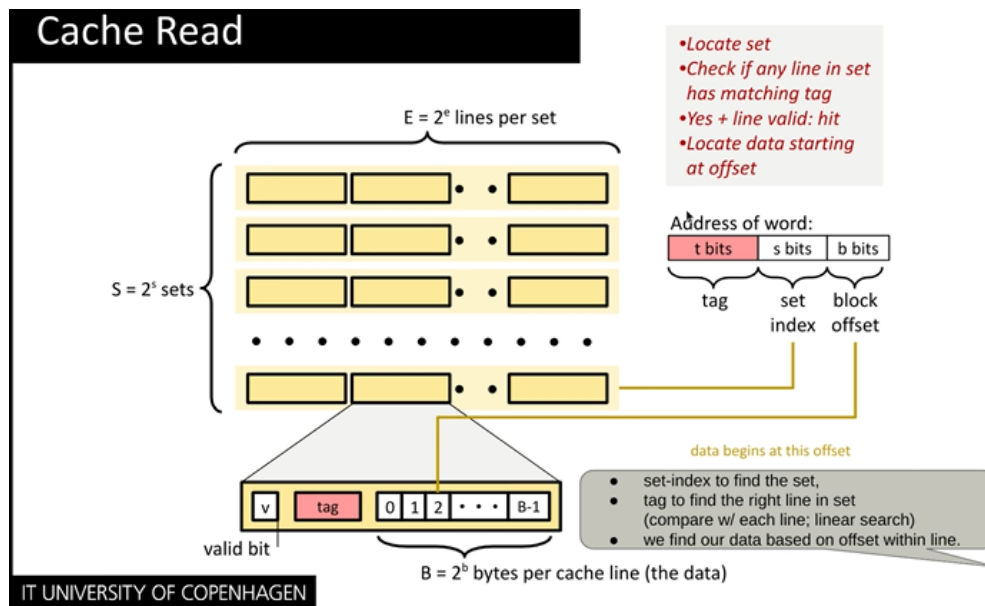


- Has some number of sets (S)

- Has some number of lines (E)
- A cache line contains



- A bit which indicates whether the line is valid (usually whether it contains up to date or stale data)
- Tag which identifies the cache line
- The actual data
- Cache size is $S * E * B$ (where B is the number of bytes of data in a cache line)
- The address of a word in the cache will contain the tag, set and block indices



Writes to memory

- **Write-through** - write immediately to memory
- **Write-back** - CPU only writes to the cache and then whatever value from memory got cached is now stale and will only be updated if the cached line

gets written back to memory - this is faster

- **Write-miss**

- **Write-allocate** - load data from memory into the cache and then update the line in the cache - good if more writes to the location follow
- **No-write-allocate** - Write immediately to memory - this is faster
- Typically what is used is either **write-through + no-write-allocate** or **write-back + write-allocate**

Cache Performance Metrics

- **Miss Rate**

- Fraction of memory references not found in cache (**misses / total accesses**) = 1 - hit rate
- Typical numbers
 - 3-10% for L1
 - <1% for L2

- **Hit Time**

- Time to deliver a line in the cache to the processor (includes time to determine if the line is in the cache)
- Typical numbers
 - 1-2 clock cycle for L1
 - 5-20 clock cycles for L2

- **Miss Penalty**

- Additional time required because of a miss
- Typically 50-200 cycles for main memory

Writing cache friendly code

- Make the common case as optimized as possible - focus on the inner loops of the core functions
- Minimize misses in the inner loops
 - Repeated references to variables (temporal locality)
 - Referencing elements one after the other ("stride 1") (spatial locality) - ex `arr[0]` and `arr[1]`

Virtual Memory

- A system using virtual addressing
- Maps virtual addresses to physical addresses
 - This is done using a **page table** - an array of page table entries that maps virtual pages to physical pages - this is a per-process data structure belonging to the kernel in DRAM
 - Some of the pages will be in physical memory (DRAM) and others on disk
 - **Page fault** - reference of virtual address that is not in physical memory (DRAM cache miss)
 - Program execution is suspended, OS takes over and places the missing data into memory

Locality to the Rescue Again!

Virtual memory **works** because of **locality**

At any point in time, programs tend to access a set of active virtual pages called the **working set**

Programs with better temporal locality will have smaller working sets

If (working set size < main memory size)

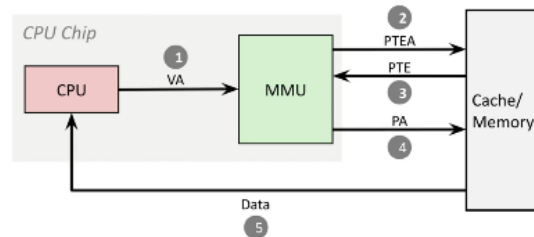
Good performance for one process after compulsory misses

If (SUM(working set sizes) > main memory size)

Thrashing: Performance meltdown where pages are swapped (copied) in and out continuously (to/from disk; "swapping"; slooow)

- Overview of the whole virtual address translation process

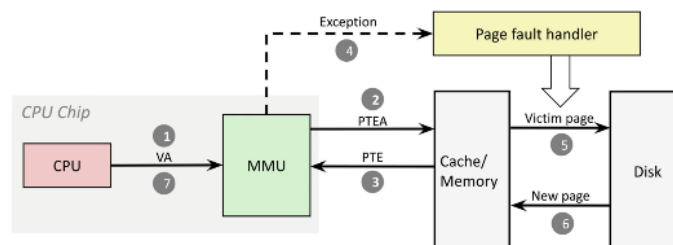
Address Translation: Page Hit



- 1) Processor sends virtual address to MMU
- 2-3) MMU fetches PTE from page table in memory
- 4) MMU sends physical address to cache/memory
- 5) Cache/memory sends data word to processor

- Overview of process when a page fault occurs

Address Translation: Page Fault



- 1) Processor sends virtual address to MMU
- 2-3) MMU fetches PTE from page table in memory
- 4) Valid bit is zero, so MMU triggers page fault exception (page not in DRAM)
- 5) Handler identifies victim (and, if dirty, pages it out to disk)
- 6) Handler pages in new page and updates PTE in memory
- 7) Handler returns to original process, restarting faulting instruction

T UNIVERSITY OF COPENHAGEN

costly operation

- Each process can have its own memory
- Uses main memory more efficiently
- Simplifies memory management
- **Isolates address spaces**
 - One process can't interfere with another process' memory
 - User program cannot access kernel information

An address space is an ordered set of contiguous addresses (non-negative integers)

Physical address space $\Rightarrow$ associated to RAM

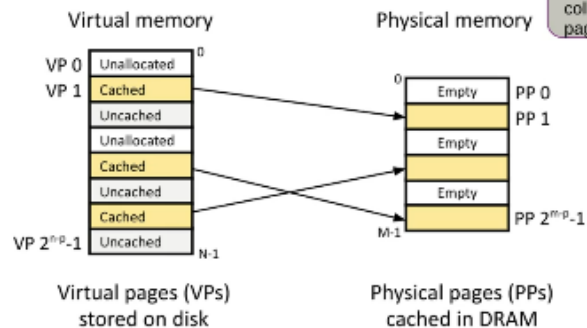
Virtual address space $\Rightarrow$ associated to each **process**

Virtual memory is an array of N contiguous bytes stored **on disk**.

The contents of the array on disk are **cached** in **physical memory** (**DRAM cache**)

These cache blocks are called *pages* (size (quanta) $P = 2^p$ bytes)

Unit of transfer between disk & virtual memory



4k, 8k, 16k
VM thus = a
collection of
pages

Q: how much VM
can we have?

- **DRAM is used for physical memory and SRAM is used for registers**

Details about modern implementations

- In modern computers, there actually are caches between the CPU and DRAM
- More details of practical implementation:

Speeding up Translation with a TLB

Page table entries (PTEs) are cached in L1 like any other memory word

PTEs may be evicted by other data references

PTE hit still requires a small L1 delay.

we don't want to pollute the L1 cache w/ PTEs

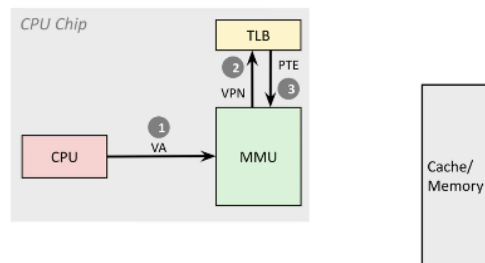
Solution: **Translation Lookaside Buffer (TLB)**

Small hardware cache in MMU

Maps virtual page numbers to physical page numbers

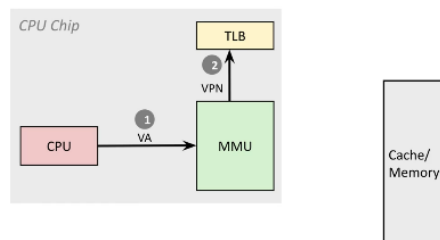
Contains complete page table entries for small number of pages

TLB Hit



A TLB hit eliminates a memory access

TLB Miss



A TLB miss incurs an additional memory access (the PTE)

Fortunately, TLB misses are rare. Why?

IT UNIVERSITY OF COPENHAGEN

- TLB misses are rare because of locality - we access elements from the same page a lot more than from different pages

